Introduction aux Bases de données

Cours 3 : Calcul Relationnel

LU2IN009

Interrogation des données

Algèbre relationnelle (voir LU3IN009)

- Langage procédural : on exprime comment répondre la requête avec des opérations
- Fondé sur la théorie des ensembles

Calcul relationnel des n-uplets

- Langage déclaratif : on exprime ce qu'on cherche (sans dire comment)
- Fondé sur la logique des prédicats

SQL (Structured Query Language)

- Sémantique basé sur le calcul relationnel (quoi)
- Exécution basée sur l'algèbre relationnel (comment)
- Comporte en plus des fonctions d'agrégation
- Implante également les opérateurs de l'algèbre

Logique des prédicats

Logique: Système formel utilisé en mathématique (preuves, théorie des ensembles), informatique (programmation, algorithmiques, IA, circuits), électronique, philosophie, droit, ...

<u>Logique propositionnelle</u>

Si Jean est malade
alors il ne travaille pas
Jean est malade
Conclusion : Jean ne travaille pas

JeanEstMalade → ¬ JeanTravaille
JeanEstMalade

¬ JeanTravaille

<u>Logique des prédicats</u>

si quelqu'un est malade
alors il ou elle ne travaille pas
Jean est malade
Conclusion : Jean travaille pas

Malade(x) → ¬ Travaille(x)
Malade(Jean)

¬ Travaille(Jean)

Calcul Relationnel des n-uplets : aperçu

Langage fondé sur la logique des prédicats

- Tuple avec n attributs → prédicat n-aire
- Valeur atomique → constante

Etudiants

Tuples

matricule	nom	prénom	dateNaiss	adresse
1753	Smith	Joe	1992-01-12	11 CP NYC
9832	Smith	Dan	1989-04-03	22 Rd NJ
7899	King	Stan	1989-05-03	27 Rd CA

Prédicats (atomes)

Etudiants(‘1753’,’Smith’, ‘Joe’, ’1992-01-12’, ‘11 CP NYC’)
Etudiants(‘9832’,’Smith’, ‘Dan’, ’ 1989-04-03’, ‘22 Rd NJ’)
…

Calcul Relationnel des n-uplets : aperçu

Langage fondé sur la logique des prédicats

- Formule logique pour exprimer une condition devant être respectée par les n-uplets à retourner
- Requête : retourner les étudiants ayant pour nom ‘Smith’
- Prédicat : Etudiants(x) ∧ x.nom=‘Smith’

Etudiants

matricule	nom	prénom	dateNaiss	adresse
1753	Smith	Joe	1992-01-12	11 CP NYC
9832	Smith	Dan	1989-04-03	22 Rd NJ
7899	King	Stan	1989-05-03	27 Rd CA

Réponse

Calcul Relationnel des n-uplets : aperçu

- Requête : retourner les étudiants inscrits en LU2IN009
- **Prédicat** : Etudiants(x) ∧ ∃ y (Inscriptions(y) ∧x.matricule=y.matricule
  ∧ y.code='LU2IN009)

Etudiants

Réponse

matricule	nom	prénom	dateNaiss	adresse
1753	Smith	Joe	1992-01-12	11 CP NYC
9832	Smith	Dan	1989-04-03	22 Rd NJ
7899	King	Stan	1989-05-03	27 Rd CA

Inscriptions

Matricule*	Code*
1753	LU2IN009
9832	LU2IN002
7899	LU2IN009

Calcul Relationnel des n-uplets : aperçu

- **Requête :** retourner les étudiants inscrits en Bases de Données ('BD')
- ***Prédicat :*** Etudiants(x) ∧ ∃y, z (Inscriptions(y) ∧ Modules(z)

  ∧ x.matricule=y.matricule ∧

  ∧ y.code=z.code∧z.intitule='BD'

Etudiants

matricule	nom	prénom	…
1753	Smith	Joe	
9832	Smith	Dan	
7899	King	Stan	

Inscriptions

Matricule*	Code*
1753	LI341
7899	LI345
9832	LI341

Modules

code	intitulé	niveau
LI341	BD	L3
LI345	Web	L3
LI399	Crypto	L3

Calcul Relationnel des n-uplets : sémantique

Forme générale des requêtes

{ Projection | Filtre }

Format résultat — Prédicat logique

Retourner les n-uplets spécifiés par Projection tel que le prédicat Filtre est vrai

- **Approche descriptive (déclarative)** : permet de décrire l'ensemble des réponses sans spécifier explicitement comment elles doivent être calculées
- A fortement influencé la conception de SQL et de Query-by-Example (QBE)

Calcul Relationnel des n-uplets : syntaxe

{ Projection | **Filtre** }

- Noms de prédicats, variables, attributs :
  - Ensemble de **variables** Var (variables n-uplets)
  - Ensemble de noms de **prédicats** Pred (tables)
  - Ensemble de noms de **d'attributs** Attr (colonnes)

- **Filtre** = prédicat logique :
  - R(v) : R $\in$ Pred, v $\in$ Var
  - v.A op w.B : op $\in \{=,\neq,>,<,\leq,\geq\}$, v,w $\in$ Var, A,B $\in$ Attr
  - v.A op 'val' : op $\in \{=,\neq,>,<,\leq,\geq\}$, v $\in$ Var, A $\in$ Attrv, 'val' = constante
  - C1 $\wedge$ C2 , C1 $\vee$ C2 , $\neg$C1, C1 $\rightarrow$ C2 : C1 et C2 sont des prédicats
  - $\exists$ v (C) , $\forall$ w (C) : v et w sont des variables libres dans le prédicat C

Variables libres et liées

- Variables **libres** : variables qui <u>ne sont pas liées</u> par un quantificateur ∃ ou ∀
- Variables **liées** : variables qui sont liées par un quantificateur ∃ ou ∀
- Réponse d'un prédicat P avec un ensemble de variable **libres** V :
  - Si V est vide, la réponse de P est vrai ou faux.
  - Si V={ v1, v2, v3, …, vn }, la réponse de P retourne tous les ensembles de nuplets { t1, t2, t3, …, tn} où P est vrai après avoir remplacé chaque variable libre vi par le nuplet ti.

Exemples :

- R(v) : v est libre → tous les nuplets de R
- R(v) ∧ v.A=3 : v est libre → tous les nuplets de R avec A=3
- ∃ v R(v) : v est liée par ∃ → vrai si R n'est pas vide (faux sinon)
- ∃ v R(v) ∧ T(w) : v est liée par ∃, w est libre → tous les nuplets de T si R n'est pas vide
- ∀ v R(v) : v est liée par ∀ → comme R est fini, il n'est pas possible que R contient tous les n-uplets possibles (toujours faux)

Calcul Relationnel des n-uplets : Sémantique

Nom	Prédicat	Sémantique
Liaison de variables	R(v)	la variable libre v est liée aux n-uplets de R
Expressions de comparaisons	v.A op w.B v.A op 'val'	vrai lorsque l'attribut A du n-uplet v est égal/différent de/… l'attribut B du n-uplet w - op $\in \{=,\neq,>,<,\leq,\geq\}$ vrai lorsque l'attribut A du n-uplet v est égal/différent de/… val - op $\in \{=,\neq,>,<,\leq,\geq\}$
Expressions booléennes	C1 $\wedge$ C2 C1 $\vee$ C2 $\neg$ C C1 $\rightarrow$ C2	vrai lorsque les deux prédicats C1 et C2 sont vrais vrai lorsqu'au moins un des deux prédicats C1 ou C2 est vrai vrai lorsque le prédicat C est faux correspond à $\neg$ C1 $\vee$ C2
Quantificateur existentiel	$\exists$ v (C)	vrai si le prédicat C est vrai pour au moins un n-uplet v (v est libre dans C)
Quantificateur universel	$\forall$ w (C)	vrai le prédicat si C est vrai pour tout les n-uplets w (w est libre dans C)

Calcul Relationnel des n-uplets : syntaxe

- Priorité des connecteurs logiques

  1. Négation $\neg$
  2. Conjonction $\wedge$
  3. Disjonction $\vee$
  4. Implication $\rightarrow$

Utilisation des parenthèses pour lever l'ambiguité :

- $\neg R(x) \wedge T(y) \vee S(z)$ correspond à $((\neg R(x)) \wedge T(y)) \vee S(z)$
- $R(x) \vee T(y) \rightarrow S(z)$ correspond à $\neg((R(x)) \vee T(y)) \vee S(z)$

Calcul Relationnel des n-uplets : Projections

{ Projection | Filtre }

Variables libres
(non-quantifiées)

Variables liées
(quantifiées)

Rappel : les variables libres dans le prédicat Filtre correspondent à des t-uplets pour lequel le prédicat est vrai.

Projections simples :

- v.A : valeur de l'attribut A du nuplet v
- v : séquence des valeurs de tous les attributs du nuplet v

Projection :

- séquence de projections simples incluant <u>toutes les variables libres</u> dans Filtre

Calcul Relationnel des n-uplets : exemples

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

1. Tous les étudiants.
2. Le prénom des étudiants ayant pour nom de famille Smith.
3. Le matricule des étudiants inscrits dans le module ‘BD’ et le code de ce module.
4. a) Les intitulés des modules où ‘Jack’ est inscrit.
   b) Les étudiants inscrits dans un même module que ‘Jack’ et le code de ce module.
5. a) Les étudiants inscrits dans au moins deux modules.
   b) Les étudiants inscrits à au moins deux modules de niveau L3.
6. a) Les étudiants qui ne sont pas inscrits au module ‘LI345’.
   b) Les étudiants qui ne sont inscrits dans aucun module.
7. a) Les étudiants ne sont pas inscrits dans tous les modules.
   b) Les étudiants sont inscrits dans tous les modules.

1. Tous les étudiants

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 1: Toutes les étudiants avec toutes leurs propriétés ?

1. Tous les étudiants

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 1: Tous les étudiants avec toutes leurs propriétés ?

$$Q(e) = \{e \mid Etudiants(e)\}$$

$$Q(e) = \{e \mid e \in Etudiants\}$$

2. Sélection

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 2: Le prénom des étudiants ayant pour nom de famille Smith ?

2. Sélection

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 2: Le prénom des étudiants ayant pour nom de famille Smith ?

Q(e) = {e.prenom | Etudiants(e) ∧ e.nom='Smith'}

Q(e) = {e.prenom | e ∈ Etudiants ∧ e.nom='Smith'}

3. Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 3: Le matricule des étudiants inscrits dans le module 'BD' ainsi que le code de ce module ?

3. Jointure

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 3: Le matricule des étudiants inscrits dans le module 'BD' ainsi que le code de ce module ?

Q(e,m) = { e.matricule, m.code | Etudiants(e)
∧ Modules(m) ∧ m.intitule='BD'
∧ **∃i** (Inscriptions(i) ∧ i.matricule=e.matricule ∧ i.code=m.code) }

3. Jointure

Etudiants

	matricule	nom	prénom	…
e→	**1753**	Smith	Joe	
	9832	Smith	Dan	
	7899	King	Stan	

Inscriptions

	Matricule*	Code*
∃i→	1753	LI341
	1753	LI345
	9832	LI341

Modules

	code	intitulé	niveau
m→	**LI341**	BD	L3
	LI345	Web	L3
	LI399	Crypto	L3

Requête 3: Le matricule des étudiants inscrits dans le module 'BD' ainsi que le code de ce module ?

Q(e,m) = {e.matricule, m.code | Etudiants(e)
∧ Modules(m) ∧ m.intitule='BD'
∧ **∃i** (Inscriptions(i) ∧ i.matricule=e.matricule ∧ i.code=m.code)}

4-a) Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 4-a) Les intitulés des modules où 'Jack' est inscrit ?

4-a) Jointure

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 4-a) Les intitulés des modules où 'Jack' est inscrit ?

Q(m) = {m.intitule | Module(m) ∧
∃e (Etudiants(e) ∧ e.prenom='Jack' ∧
∃i (Inscriptions(i) ∧ i.matricule=e.matricule ∧ i.code=m.code)) }

4-a) Jointure

Etudiants

	matricule	nom	prénom	…
	1753	Smith	Joe	
∃e→	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

	matricule*	code*
	1753	LI341
	1753	LI345
∃i→	9832	LI341

Modules

	code	intitulé	niveau
m→	LI341	**BD**	L3
	LI345	Web	L3
	LI399	Crypto	L3

Requête 4-a) Les intitulés des modules où 'Jack' est inscrit ?

Q(m) = {**m.intitule** | Module(m)
∧ **∃e**(Etudiants(e) ∧ e.prenom='Jack'
∧ **∃i** (Inscriptions(i) ∧ i.matricule=e.matricule ∧ i.code=m.code)) }

4-b) Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 4-b) Le matricule des étudiants inscrits dans un même module que 'Jack' ainsi que l'intitulé de ces modules ?

4-b) Jointure

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 4-b) Le matricule des étudiants inscrits dans un même module que 'Jack' ainsi que l'intitulé de ces modules ?

Q(e, m) = {e.matricule, m.intitule | Etudiants(e) ∧ Module(m)
∧ ∃f (Etudiants(f) ∧f.prenom='Jack'
∧ ∃i1,i2 (Inscriptions(i1) ∧ Inscriptions(i2)
∧ i1.matricule=e.matricule ∧ i2.matricule=f.matricule
∧ i1.code=i2.code ∧ m.code=i1.code)) }

4-b) Jointure

Etudiants

	matricule	nom	prénom	…
e→	**1753**	Smith	Joe	
∃f→	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

	matricule*	code*
∃i1→	1753	LI341
	1753	LI345
∃i2→	9832	LI341

Modules

	code	intitulé	niveau
m→	LI341	**BD**	L3
	LI345	Web	L3
	LI399	Crypto	L3

Requête 4-b) Le matricule des étudiants inscrits dans un même module que 'Jack' ainsi que l'intitulé de ces modules ?

Q(e, m) = {e.matricule, m.intitule | Etudiants(e) ∧ Module(m)
∧ ∃f (Etudiants(f) ∧f.prenom='Jack'
∧ ∃i1,i2 (Inscriptions(i1) ∧ Inscriptions(i2)
∧ i1.matricule=e.matricule ∧ i2.matricule=f.matricule
∧ i1.code=i2.code ∧ m.code=i1.code)) }

5-a) Auto-Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 5-a: Le matricule des étudiants inscrits dans au moins deux modules ?

5-a) Auto-Jointure

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 5-a: Le matricule des étudiants inscrits dans <u>au moins deux</u> modules ?

Q(e) = {e.matricule | Etudiants(e)
∧ **∃i1, i2** (Inscriptions(i1) ∧ Inscriptions(i2)
∧ i1.matricule=e.matricule ∧ i2.matricule=e.matricule
∧ i1.code ≠ i2.code) }

5-a) Auto-Jointure

Etudiants

	matricule	nom	prénom	...
e→	**1753**	Smith	Joe	
	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

	matricule*	code*
∃i1→	1753	LI341
∃i2→	1753	LI345
	9832	LI341

Modules

code	intitulé	niveau
LI341	BD	L3
LI345	Web	L3
LI399	Crypto	L3

Requête 5-a): Le matricule des étudiants inscrits dans au moins deux modules ?

Q(e) = {e.matricule | Etudiants(e)

∧ **∃i1, i2** (Inscriptions(i1) ∧ Inscriptions(i2)

∧ i1.matricule=e.matricule ∧ i2.matricule=e.matricule

∧ **i1.code ≠ i2.code**) }

5-b) Auto-Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 5-b: Le matricule des étudiants inscrits dans au moins deux modules de niveau L3?

5-b) Auto-Jointure

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 5-b: Le matricule des étudiants inscrits dans au moins deux modules de niveau L3?

Q(e) = { e.matricule | Etudiants(e)

∧ ∃i1, i2 (Inscriptions(i1) ∧ Inscriptions(i1)

∧ i1.matricule=e.matricule ∧ i2.matricule=e.matricule

∧ i1.code ≠ i2.code ∧

∃m1, m2 (Module(m1) ∧ Module(m2)

∧ m1.code=i1.code ∧ m2.code=i2.code

∧ **m1.niveau='L3'** ∧ **m2.niveau='L3'**)) }

5-b) Auto-Jointure

Etudiants

	matricule	nom	prénom	…
e→	**1753**	Smith	Joe	
	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

	matricule*	code*
∃i1→	1753	LI341
	1753	M009
	9832	LI341
∃i2→	1753	LI345

Modules

	code	intitulé	niveau
∃m1→	LI341	BD	L3
∃m2→	LI345	Web	L3
	LI399	Crypto	L3
	M009	BD	M1

Q(e) = {e.matricule | Etudiants(e)

∧ **∃i1, i2** (Inscriptions(i1) ∧ Inscriptions(i2)

∧ i1.matricule=e.matricule

∧ i2.matricule=e.matricule

∧ i1.code ≠ i2.code

∧ **∃m1, m2** (Module(m1) ∧ Module(m2)

∧ m1.code=i1.code ∧ m2.code=i2.code

∧ m1.niveau='L3' ∧ m2.niveau='L3')) }

6-a) Différence

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

- Requête 6-a) Les étudiants qui ne sont pas inscrits au module ‘LI345’?

6-a) Différence : ¬ ∃

Etudiants

	matricule	nom	prénom	...
	1753	Smith	Joe	
e→	**9832**	Smith	Jack	
	7899	King	Stan	

Inscriptions

matricule*	code*
1753	LI341
1753	LI345
9832	LI341

Requête 6-a) Les étudiants qui ne sont pas inscrits au module 'LI345'?

Q(e) = {e.matricule | Etudiants(e) ∧
¬ ∃ i (Inscriptions(i) ∧ i.matricule=e.matricule
∧ i.code = 'LI345') }

Les tuples (9832,LI345) et (7899,LI345) n'existent pas dans Inscriptions

6-a) Différence : ¬ ∃

Etudiants

	matricule	nom	prénom	...
e→	**1753**	Smith	Joe	
	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

	Matricule*	Code*
i →	1753	LI341
	1753	LI345
	9832	LI341

Requête 6-a) Les étudiants qui ne sont pas inscrits au module 'LI345'?

Q(e) = {e.matricule | Etudiants(e)
∧ ∃ i (Inscriptions(i) ∧ i.matricule=e.matricule
∧ i.code ≠ 'LI345') }

Réponse incorrecte

Qu'est-e que cette requête retourne ?

6-b) Différence

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 6-b) Les étudiants qui ne sont inscrits dans aucun module?

6-b) Différence

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 6-b) Les étudiants qui <u>ne sont inscrits dans aucun</u> module?

Q(e) = {e.matricule | Etudiants(e)
∧ ¬∃i (Inscriptions(i) ∧ i.matricule=e.matricule) }

Q(e) = {e.matricule | Etudiants(e)
∧ ¬∃m(Module(m) ∧ ∃i (Inscriptions(i)
∧ i.matricule=e.matricule ∧ m.code=i.code)) }

6-b) Différence

Etudiants

	matricule	nom	prénom	...
	1753	Smith	Joe	
e →	**9832**	Smith	Jack	
	7899	King	Stan	

Inscriptions

Matricule*	Code*
1753	LI341
1753	LI345

Modules

code	intitulé	niveau
LI341	BD	L3
LI345	Web	L3

Requête 6-b) Les étudiants qui ne sont inscrits dans aucun module?

Q(e) = {e.matricule | Etudiants(e)
∧ ¬∃i (Inscriptions(i) ∧ i.matricule=e.matricule) }

7-a) Différence

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 7-a): Les étudiants ne sont pas inscrits dans tous les modules ?

7-a) Différence

Etudiants

	matricule	nom	prénom	…
	1753	Smith	Joe	
e →	**9832**	Smith	Jack	
	7899	King	Stan	

Inscriptions

matricule*	mode*
1753	LI341
1753	LI345
9832	LI345

Modules

	code	intitulé	niveau
∃m →	LI341	BD	L3
	LI345	Web	L3

Requête 7-a): Les étudiants <u>ne sont pas inscrits dans tous les</u> modules ?

Q(e) = {e.matricule | Etudiants(e)
∧ ∃m(Module(m) ∧ ¬∃i (Inscriptions(i)
∧ i.matricule=e.matricule ∧ m.code=i.code)) }

7-b) Différence et Division

Etudiants(matricule, nom, prenom, adresse, collaborateur*)

Inscriptions(matricule*, code*)

Modules(code, intitule, niveau, salle*)

Salles(numero, capacite, précédent *, suivante*)

Requête 7-b): Les étudiants qui sont inscrits dans tous les modules?

7-b) Différence et Division

Etudiants(<u>matricule</u>, nom, prenom, adresse, collaborateur*)

Inscriptions(<u>matricule*, code*</u>)

Modules(<u>code</u>, intitule, niveau, salle*)

Salles(<u>numero</u>, capacite, précédent *, suivante*)

Requête 7-b): Les étudiants qui <u>sont inscrits dans tous les</u> modules ?

Q(e) = {e.matricule | Etudiants(e)
∧ **¬∃m**(Module(m)
∧ ¬∃i (Inscriptions(i)
∧ i.matricule=e.matricule ∧ m.code=i.code))) }

7-b) Différence et Division

Etudiants

	matricule	nom	prénom	...
e →	**1753**	Smith	Joe	
	9832	Smith	Jack	
	7899	King	Stan	

Inscriptions

matricule*	code*
1753	LI341
1753	LI345
9832	LI345

Modules

code	intitulé	niveau
LI341	BD	L3
LI345	Web	L3

Requête 7-b: Les étudiants inscrits <u>dans tous les</u> modules ?

Q(e) = {e.matricule | Etudiants(e)
∧ **¬∃m**(Module(m) ∧ ¬∃i (Inscriptions(i)
∧ i.matricule=e.matricule ∧ m.code=i.code)) }

Complément de la requête 7-a : Tous les étudiants sauf ceux qui ne sont pas inscrits dans tous les modules ?

Cas particuliers

- Requêtes sans réponse (réponses vides)
  - { e | Etudiants(e) ∧ Modules (e) } : deux tables ne peuvent avoir exactement le même n-uplet (schéma différents)
  - {e |Etudiants(e) ∧ e.salaire>100} : salaire n'est pas un attribut de Etudiants
  - {e |Etudiants(e) ∧ e.prenom='Jack' ∧ e.prenom='John' } : un étudiant ne peut pas avoir deux noms différents (valeur atomique)
- Requête est "non sûre" (réponses infinirs)
  - {e | ¬ Etudiants(e) } : cherche dans un domaine infini (tout sauf dans la table Etudiants)
  - {e | ∀ x Etudiants(x) ∧ … } : tous les n-uplets du monde doivent être dans Etudiant
  - {e | e.att>5} : e instanciée un nombre infini car pas liée à une table
- Pour éviter des requêtes "non sûres" :
  - introduire toute nouvelle variable x par un prédicat R(x) sans négation
  - Utiliser uniquement la quantification existentielle ∃ (on peut tout exprimer sans quantification universelle ∀)

Conclusion

Présentation d'un langage de requête fondé sur la Logique des Prédicats:

- Les prédicats sont des tables, les constantes sont les valeurs atomiques, les variables sont liées aux n-uplets
- Exprimer une requête = spécifier une prédicat logique
  → langage déclaratif
- Autre variante : les variables liées aux attributs (calcul du domaine)
- Avantage calcul des n-uplets : traduction quasi-directe vers SQL